

Preprocessing

Il **preprocessing** è la fase in cui trasformi i dati grezzi (spesso sporchi e disordinati) in un formato che un algoritmo di Machine Learning può effettivamente "digerire".

Come si dice in gergo: *Garbage In, Garbage Out* (se inserisci spazzatura, otterrai previsioni spazzatura).

Qui di seguito presentiamo i 4 pilastri fondamentali di una corretta pipeline di preprocessing.

N.B. è importante realizzare sempre preliminarmente una analisi esplorativa dei dati iniziali per "iniziare a conoscere il dataset" e capire meglio "come muoversi".

1. Pulizia dei Dati (Data Cleaning)

Prima di tutto, bisogna gestire le imperfezioni del dataset.

- **Gestione dei missing values:** Come abbiamo visto, puoi rimuovere le righe (`dropna`) o imputare i valori (`SimpleImputer`, `IterativeImputer`).
- **Rimozione dei duplicati:** Righe identiche che possono sovraccaricare l'importanza di certi pattern.
- **Gestione degli Outlier:** Valori anomali (es. un'età di 200 anni) che possono distorcere i calcoli statistici, specialmente per modelli sensibili come la regressione lineare.

2. Trasformazione delle Variabili Categoricali

Gli algoritmi matematici lavorano con i numeri, non con le parole. Dobbiamo convertire le etichette testuali:

- **Ordinal Encoding:** Se esiste un ordine logico (es. "Basso", "Medio", "Alto" diventa 1, 2, 3).
- **One-Hot Encoding:** Se non c'è ordine (es. "Colore: Rosso, Blu"). Si creano nuove colonne binarie (0 o 1) per ogni categoria, evitando che il modello pensi che "Blu" sia "maggiore" di "Rosso".

3. Scaling delle Feature (Normalizzazione e Standardizzazione)

Molti algoritmi (come KNN, SVM o Reti Neurali) misurano la "distanza" tra i dati. Se una colonna va da 0 a 1 (es. tasso di clic) e un'altra da 0 a 100.000 (es. reddito), la seconda dominerà completamente la prima.

- **StandardScaler:** Trasforma i dati in modo che abbiano media 0 e deviazione standard 1 (ideale per dati con distribuzione a campana).
- **MinMaxScaler:** Schiaccia tutti i valori in un intervallo fisso, solitamente .

4. Selezione e Ingegneria delle Feature (Feature Engineering)

Non tutti i dati sono utili. A volte meno è meglio:

- **Selezione:** Eliminare colonne ridondanti o non correlate al target (es. l'ID dell'utente non serve a predire se comprerà un prodotto).
- **Creazione:** Combinare variabili esistenti per crearne di nuove più informative (es. dividere "Area" per "Numero Stanze" per ottenere la "Grandezza Media Stanza").

Lo schema logico (Workflow)

In Python, il modo professionale per gestire tutto questo è usare le **Pipeline** di Scikit-Learn, che assicurano che le trasformazioni applicate al training set siano identiche a quelle applicate ai nuovi dati (test set).

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

# Esempio di mini-pipeline
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')), # 1. Gestione buchi
    ('scaler', StandardScaler())                 # 2. Pareggio scale
])
```

✓ Il problema dei dati mancanti

Un dataset reale presenta spesso dei dati mancanti.

La gestione di questi dati mancanti è uno dei principali problemi dell'analisi preliminare dei dati.

In presenza di dati mancanti potremmo chiederci: cosa sarebbe accaduto se i dati fossero stati completi?

Lo scopo del ricercatore è trovare metodi di analisi o di imputazione dei dati mancanti appropriati al caso sotto studio. La scelta di metodi appropriati dipende dal pattern dei dati mancanti e soprattutto dal meccanismo generatore dei dati mancanti. La classificazione standard del meccanismo generatore dei dati mancanti è: – missing

completely at random (MCAR), cioè il motivo per cui i dati sono mancanti non dipende dai dati osservati o non osservati;

– missing at random(MAR), ovvero la ragione dei dati mancanti può essere spiegata dai dati osservati;

– missing not at random(MNAR), cioè il motivo delle osservazioni mancanti è da attribuire ad osservazioni non osservate.

Per i più curiosi e Per approfondire davvero:

- Little, R. J. A. Rubin, D. B., (2002) Statistical analysis with missing data, 2nd ed.
- Wiley. Rubin DB (1976) Inference and missing data. Biometrika, 72:359-364.

Vediamo possibili approcci operativi:

Per risolvere questo "problema" ci sono diversi approcci.

E' importante conoscere quali sono gli algoritmi che non possano gestire i dati mancanti e quali possono "girare" anche in presenza di dati mancanti.

Dal punto di vista della **libreria Scikit-learn** si veda qui:

- <https://scikit-learn.org/stable/modules/impute.html>
- https://feature-engine.trainindata.com/en/1.8.x/api_doc/imputation/index.html

Per un **approfondimento teorico** ma relativamente divulgativo, invece, qui:

- <https://medium.com/@ajayverma23/data-imputation-a-comprehensive-guide-to-handling-missing-values-b5c7d11c3488>

****Per approfondire in modo rigoroso: **** Segnaliamo un articolo presente in letteratura di review del problema dei missing data.

https://www.scirp.org/pdf/jcc20241211_41732927.pdf

Ne presentiamo alcuni (elenco non esaustivo)

1. **Eliminazione di dati mancanti.** Se in una riga del dataset compaiono dati mancanti una soluzione è quella di rimuovere quella riga. Ovviamente questa soluzione ha senso solo se la percentuale di dati mancanti è piccola rispetto alla dimensione del dataset.

Esempio:

```
import pandas as pd
df = pd.read_csv('data.csv')

# Rimuove le righe che hanno almeno un valore mancante
df_clean = df.dropna()
```

```
# Rimuove le colonne che hanno valori mancanti
df_cols_clean = df.dropna(axis=1)
```

2. **Imputazione statistica semplice** con valori come la media, la moda o la mediana Questo metodo consiste nel sostituire il vuoto con un valore calcolato dal resto della colonna. È la tecnica più comune ma si presta a diverse criticità

```
# Riempie con la media
df['eta'] = df['eta'].fillna(df['eta'].mean())

# Riempie con la moda (essendo una serie, prendiamo il primo elemento [0])
df['citta'] = df['citta'].fillna(df['citta'].mode()[0])
```

3. **Imputazione Iterativa e Multivariata** Esistono in letteratura una ampia serie di metodi utilizzati per imputare i "missing data". A volte la media è troppo generica. Se un dato manca, potremmo stimarlo guardando le altre caratteristiche di quella riga. Scikit-Learn offre strumenti potenti per questo: KNN Imputer: Trova i k vicini più simili alla riga con il dato mancante e ne fa la media. IterativeImputer: Modella ogni colonna con dati mancanti come funzione delle altre colonne. Esempio con KNN:

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=5)
df_imputed = imputer.fit_transform(df)
```

✓ Un'altra possibilità

L'**IterativeImputer** di Scikit-Learn è uno dei metodi più sofisticati per gestire i dati mancanti. A differenza della media o della mediana, che guardano solo la singola colonna, questo approccio tratta ogni colonna con valori mancanti come un **problema di regressione**.

Come funziona (Il concetto di "Round Robin")

Immagina di avere tre colonne: Età, Reddito ed Esperienza. Se manca un valore nel "Reddito", l'algoritmo:

1. Usa le altre colonne (Età ed Esperienza) come variabili predittive ().
2. Usa la colonna con i buchi (Reddito) come target ().
3. Addestra un modello di regressione sui dati presenti per **predire** quelli mancanti.

Questo processo avviene in modo iterativo su tutte le colonne, "aggiustando" le stime round dopo round finché i valori non si stabilizzano.

Codice Python per applicarlo

Per utilizzare `IterativeImputer`, dobbiamo abilitarlo esplicitamente poiché è ancora considerato in fase "sperimentale" (sebbene sia stabilissimo).

```
import pandas as pd
import numpy as np
# È necessario importare questo per abilitare IterativeImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import BayesianRidge

# 1. Creiamo un dataset di esempio con valori mancanti
data = {
    'Eta': [25, 30, np.nan, 45, 50],
    'Esperienza': [2, np.nan, 7, 15, 20],
    'Reddito': [25000, 32000, 45000, np.nan, 80000]
}
df = pd.DataFrame(data)

# 2. Inizializziamo l'imputer
# estimator: il modello usato per predire (BayesianRidge è il default)
# max_iter: quante volte ripetere il ciclo di predizione
imputer = IterativeImputer(estimator=BayesianRidge(), max_iter=10, random_state=

# 3. Appliciamo l'imputazione
# Nota: restituisce un array numpy, quindi lo riconvertiamo in DataFrame
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

print("Dataset originale:")
print(df)
print("\nDataset dopo IterativeImputer:")
print(df_imputed)
```

Quando usarlo (e quando no)

- **PRO:** È estremamente accurato se esiste una correlazione tra le variabili (es. se l'esperienza aumenta, è probabile che anche il reddito aumenti).
- **CONTRO:** È computazionalmente più costoso della semplice media. Se hai milioni di righe e centinaia di colonne, potrebbe essere lento.

- **ATTENZIONE:** Funziona solo con dati **numerici**. Se hai dati categorici (testo), dovresti prima convertirli in numeri (es. con `OrdinalEncoder`).

Piccola nota tecnica: Di default usa la regressione Bayesiana (`BayesianRidge`), ma puoi passargli quasi ogni stimatore di Scikit-Learn, persino un `RandomForestRegressor` se sospetti relazioni non lineari tra i tuoi dati.

Inizia a programmare o genera codice con l'IA.